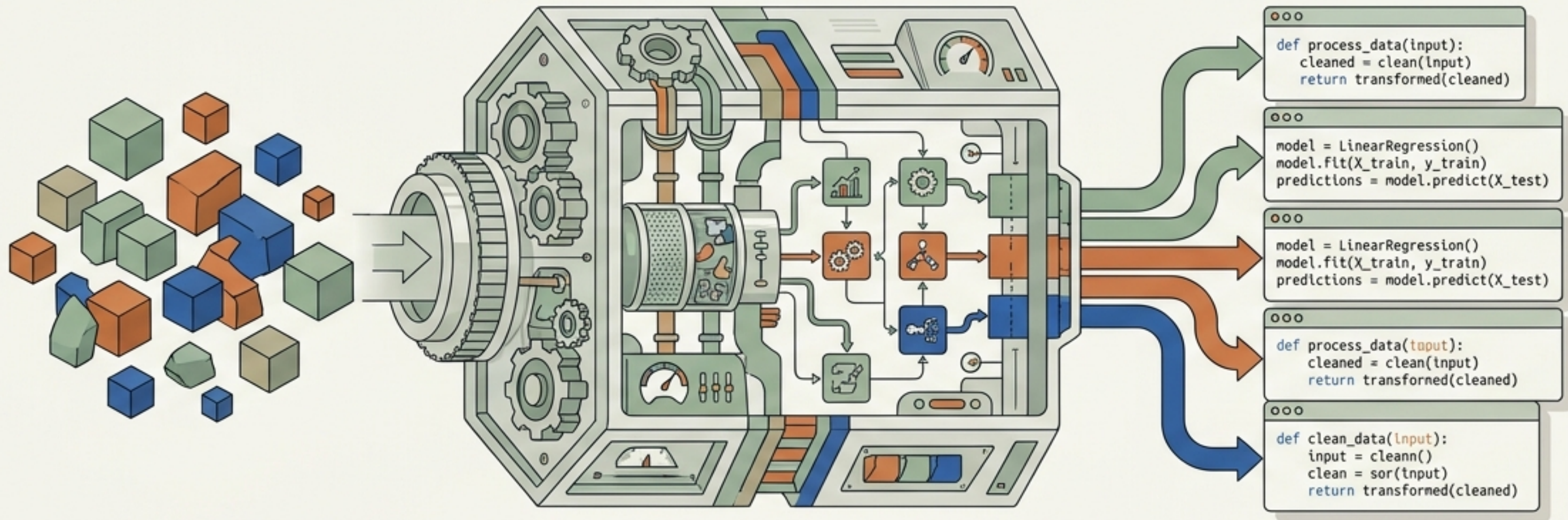




Machine Learning & KI-Algorithmen

Ein Master-Playbook: Von der mathematischen Theorie zur
Business-Implementierung.

Architektur • Evaluierung • Code-Synthese

Die Drei KI-Lernparadigmen



Überwachtes Lernen (Supervised)

Prinzip

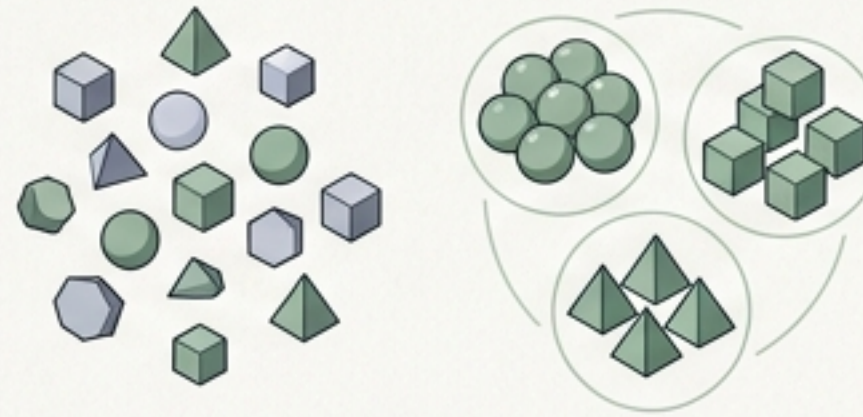
Modell lernt aus markierten (gelabelten) Beispieldaten, um eine Zielvariable vorherzusagen.

Ziel

Klassifikation (Kategorien) oder Regression (kontinuierliche Werte).

Use Case

Churn-Prediction, Umsatz-Forecast



Unüberwachtes Lernen (Unsupervised)

Prinzip

Modell analysiert unstrukturierte, ungelabelte Daten, um eigenständig verborgene Muster zu finden.

Ziel

Clustering (Gruppenbildung) oder Dimensionsreduktion

Use Case

Kundensegmentierung, Anomalieerkennung



Bestärkendes Lernen (Reinforcement)

Prinzip

Ein Agent interagiert mit einer Umgebung und lernt via Trial-and-Error durch ein Belohnungssystem (Feedback-Schleife).

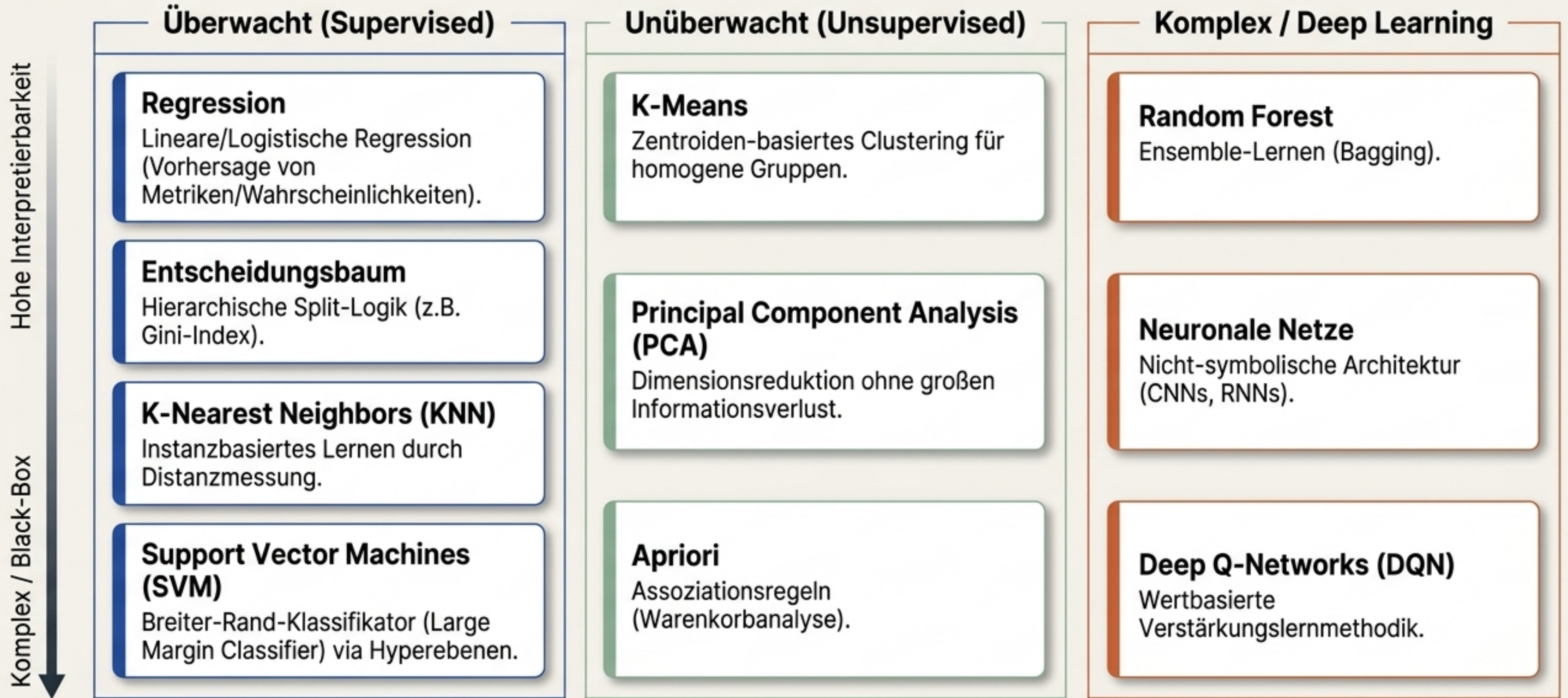
Ziel

Maximierung der kumulativen Belohnung durch autonome Strategiebildung.

Use Case

Dynamic Pricing, Robotik-Steuerung

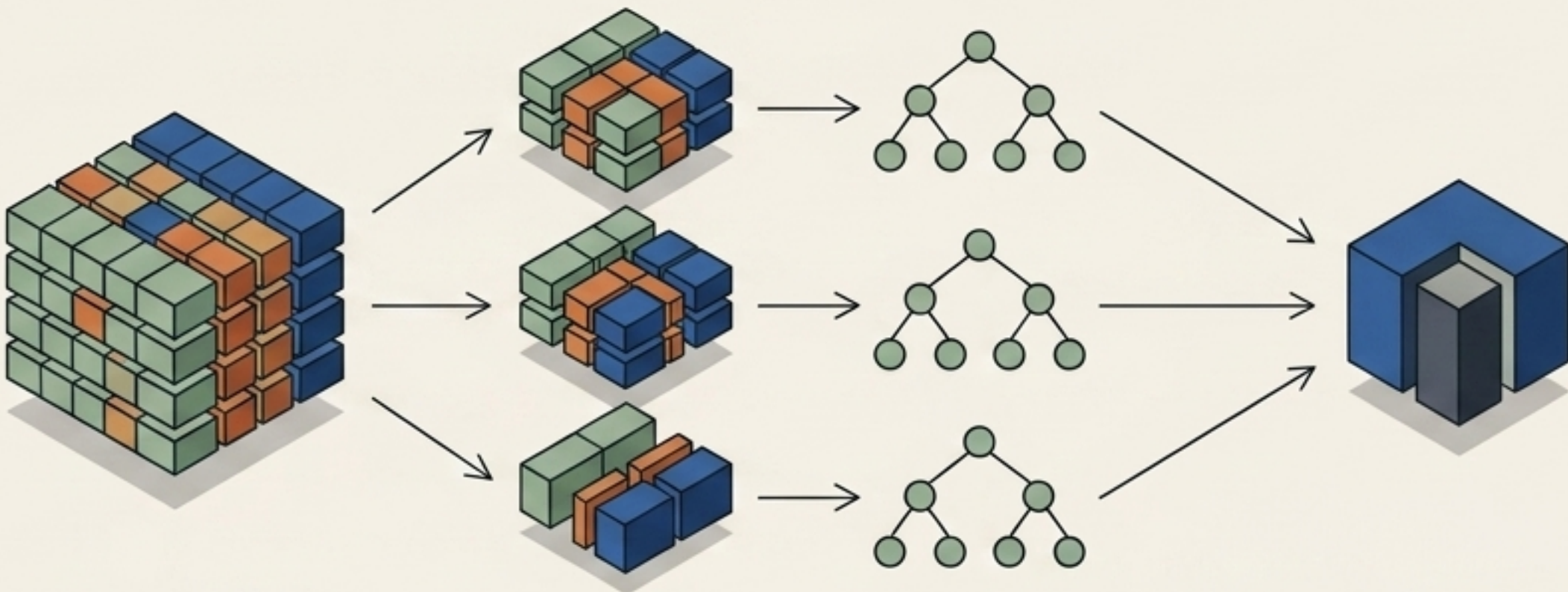
Das ML-Algorithmen-Portfolio



Vom Entscheidungsbaum zum Random Forest



Entscheidungsbaum: Hohe Varianz



Bootstrap-Aggregation / Bagging

Random Forest: Stabile Prognose

Entscheidungsbaum

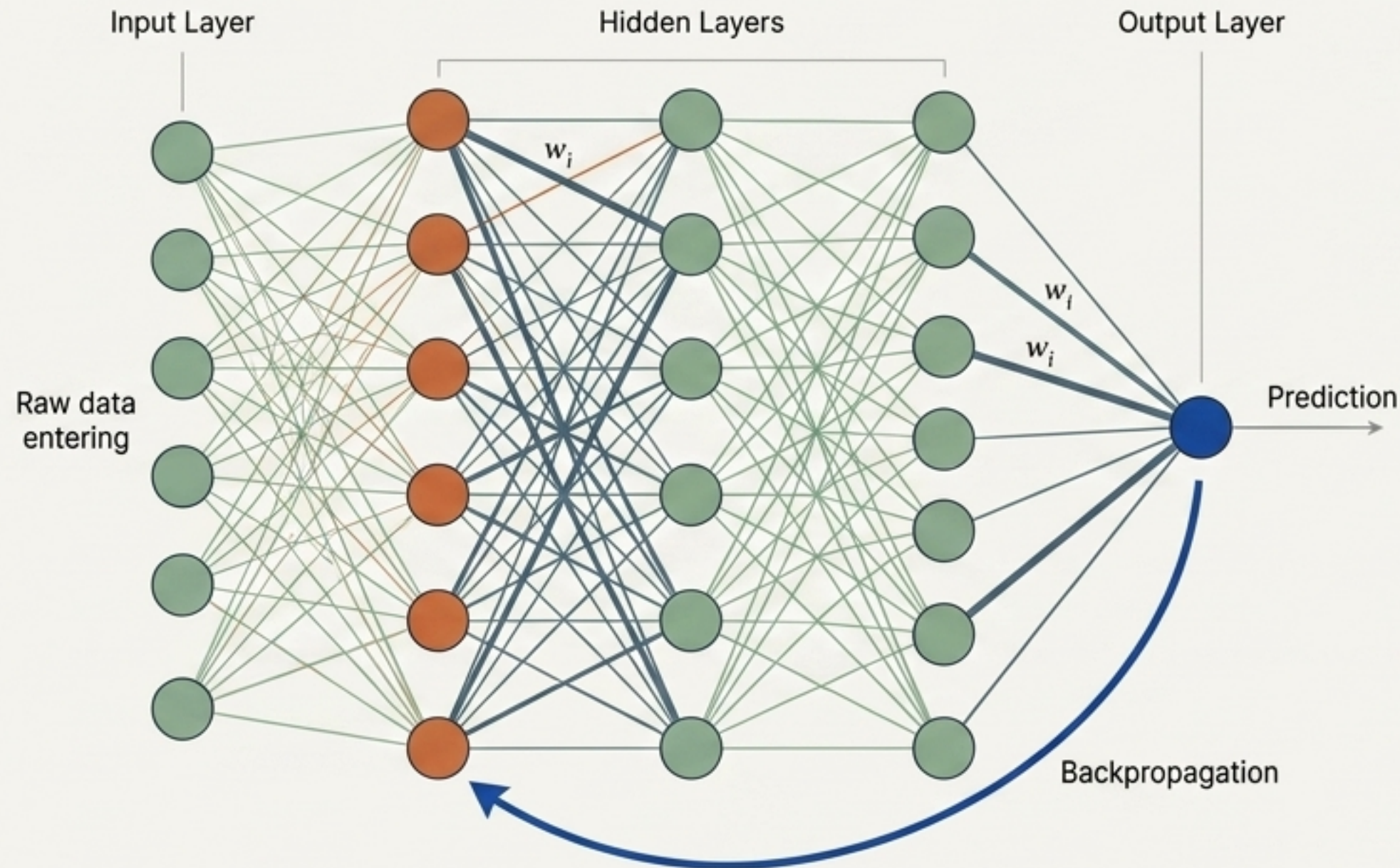
Nutzt rekursives binäres Splitting (Maximierung des Informationsgewinns/Gini-Index).
Stark anfällig für Overfitting bei Rauschen in Trainingsdaten.

Random Forest Architektur - Ein Ensemble-Verfahren

	Bagging (Bootstrap Aggregation) Erzeugt unkorrelierte Bäume durch paralleles Training auf Zufallsstichproben (mit Zurücklegen).		Mehrheitsentscheidung Aggregation der Ergebnisse minimiert die Varianz und schützt vor Überanpassung.
	Feature-Zufälligkeit Betrachtet pro Split nur eine zufällige Teilmenge an Merkmalen.		Out-of-Bag (OOB) Interne Kreuzvalidierung mit dem ungenutzten Drittel der Daten.

Neuronale Netze: Architektur des Deep Learning

Neural Concept Schematic



Das Blackbox-Modell

Inspiziert von biologischen Gehirnen verarbeiten künstliche neuronale Netze (KNN) strukturierte und unstrukturierte Daten (Bilder, Text).

Funktionsweise

- **Schichten (Layers):** Signale wandern von Eingabe- über verborgene (Hidden) zu Ausgabeschichten.
- **Gewichtung & Aktivierungsfunktion:** Reize werden gewichtet summiert. Übersteigt der Wert einen Schwellenwert, "feuert" das Neuron.
- **Backpropagation:** Der Algorithmus berechnet Gradienten der Verlustfunktion und passt Gewichte rückwärts an, um den Fehler zu minimieren.

Varianten

Convolutional Neural Networks (CNNs) für Bild-/Mustererkennung; Recurrent Neural Networks (RNNs) für Zeitreihen.

Deep-Q-Networks (DQN): Reinforcement Learning in Aktion

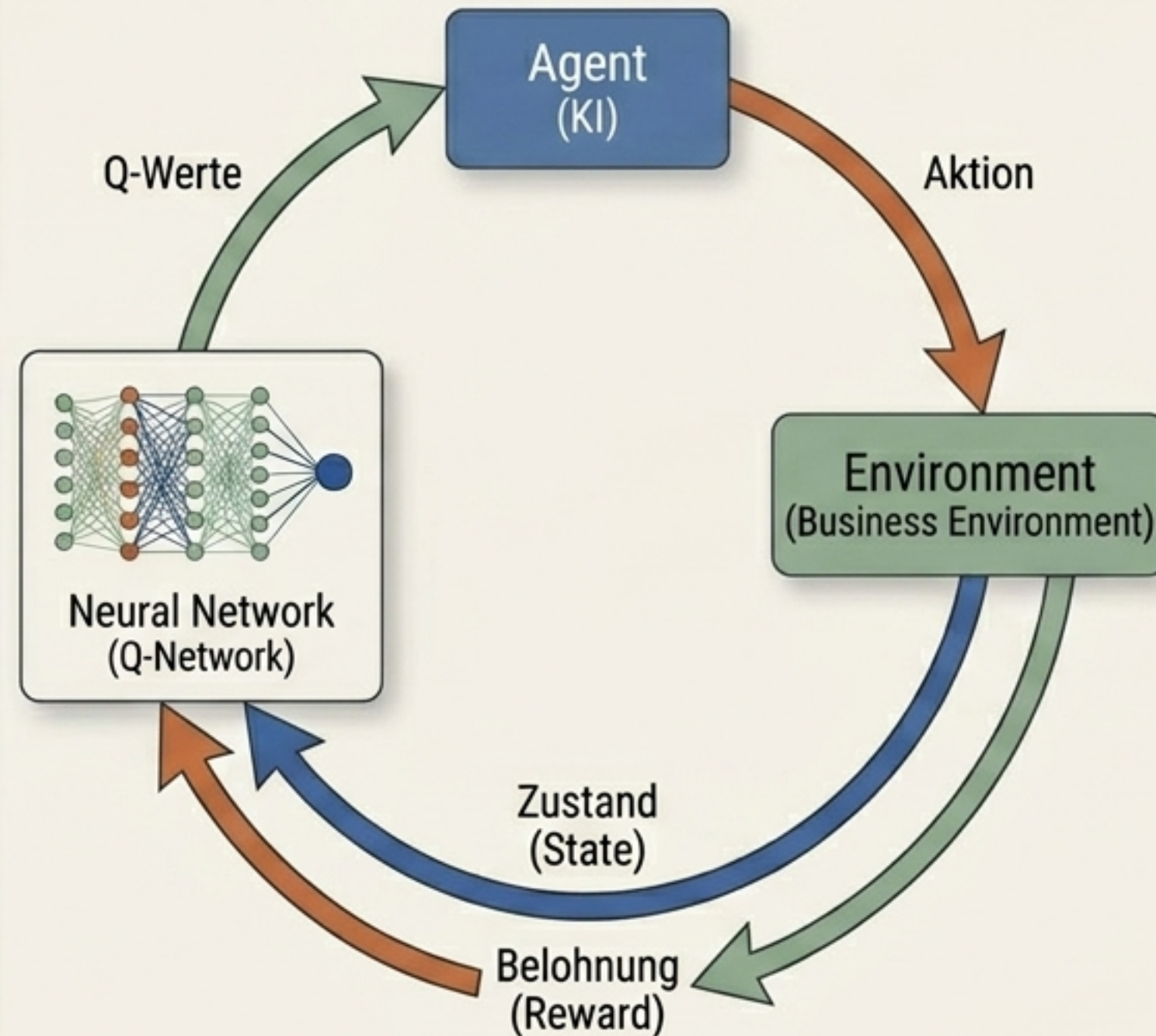
Das Konzept

DQN kombiniert klassisches Q-Learning (wertbasiert) mit tiefen neuronalen Netzen, um den erwarteten Nutzen (Q-Wert) von Aktionen in hochdimensionalen Umgebungen zu schätzen.

Zentrale Stabilitätsmechanismen

- **Experience Replay**
Speichert vergangene Übergänge (Zustand, Aktion, Belohnung, Folgezustand) in einem Puffer. Bricht zeitliche Korrelationen durch zufälliges Sampling.
- **Target Networks**
Verwendet ein separates, eingefrorenes Zielnetzwerk zur Stabilisierung der Wertaktualisierungen während des Trainings.

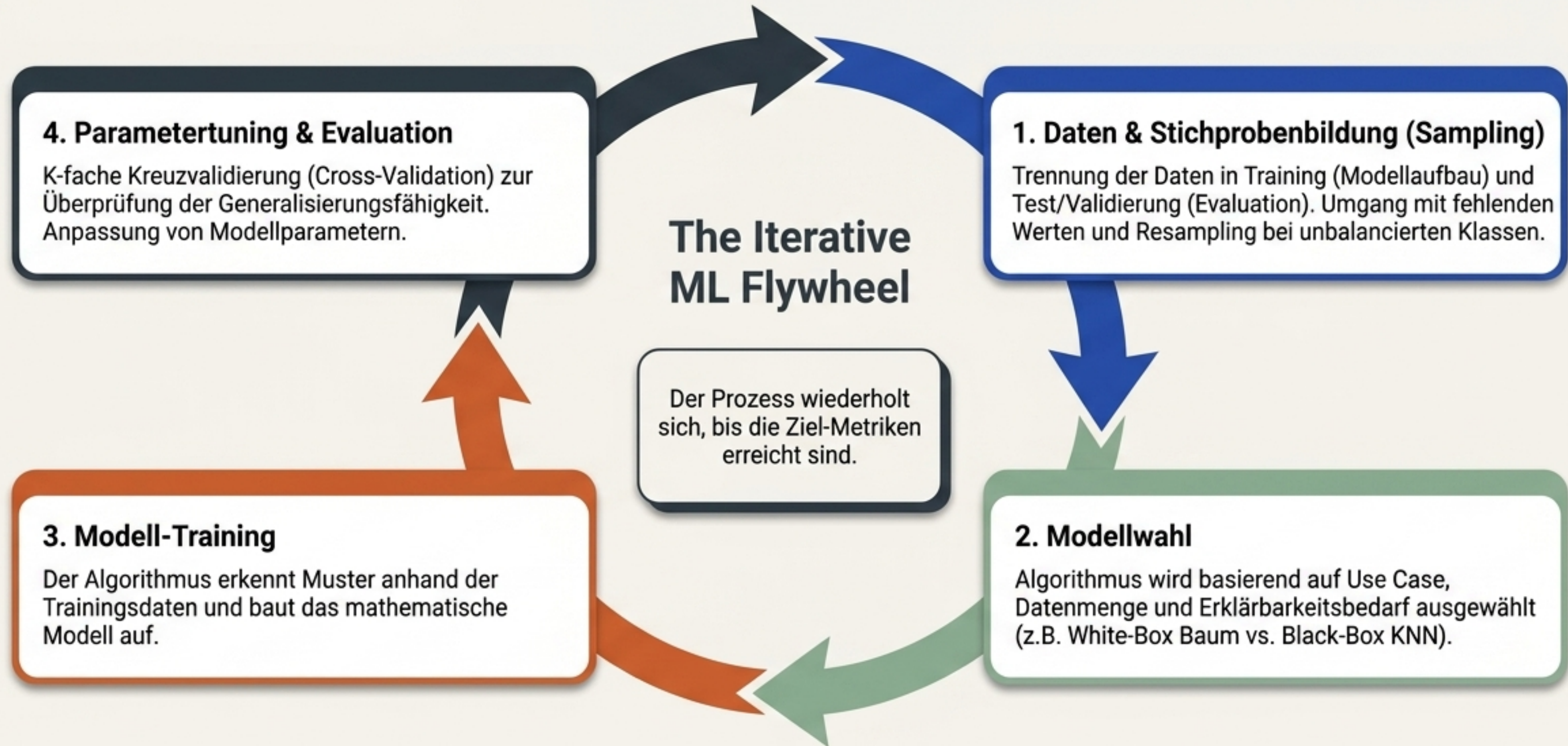
The DQN Architecture Loop



Business-Relevanz

Perfekt für sequentielle Entscheidungsprobleme wie Dynamic Pricing oder Robotik-Steuerung, bei denen diskrete Handlungen Belohnungssignale auslösen.

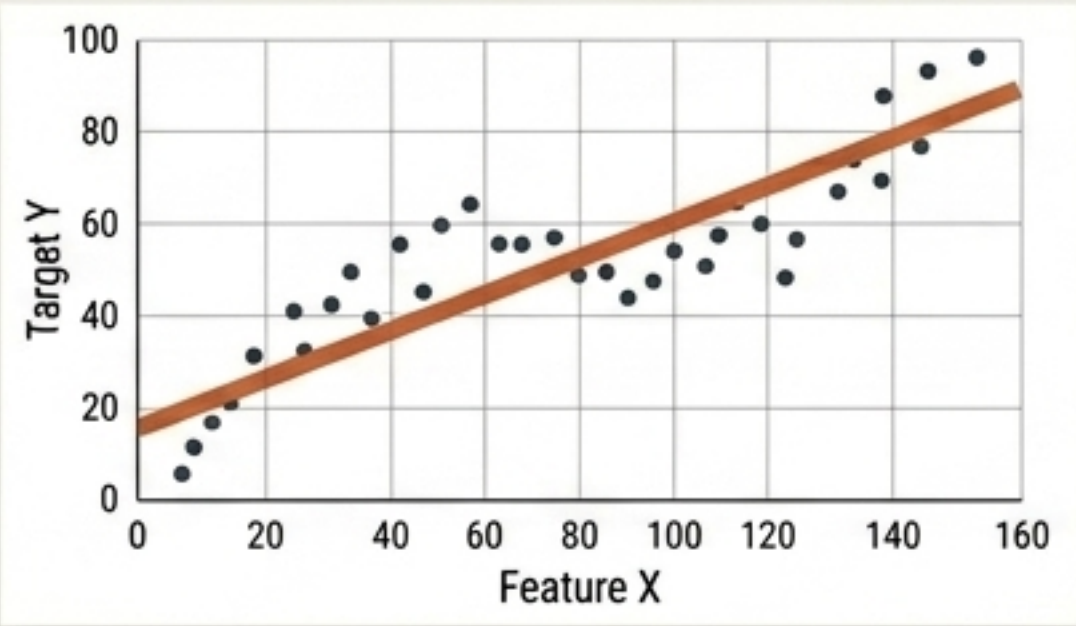
Das KI-Vorgehensmodell: Ein iterativer Lifecycle



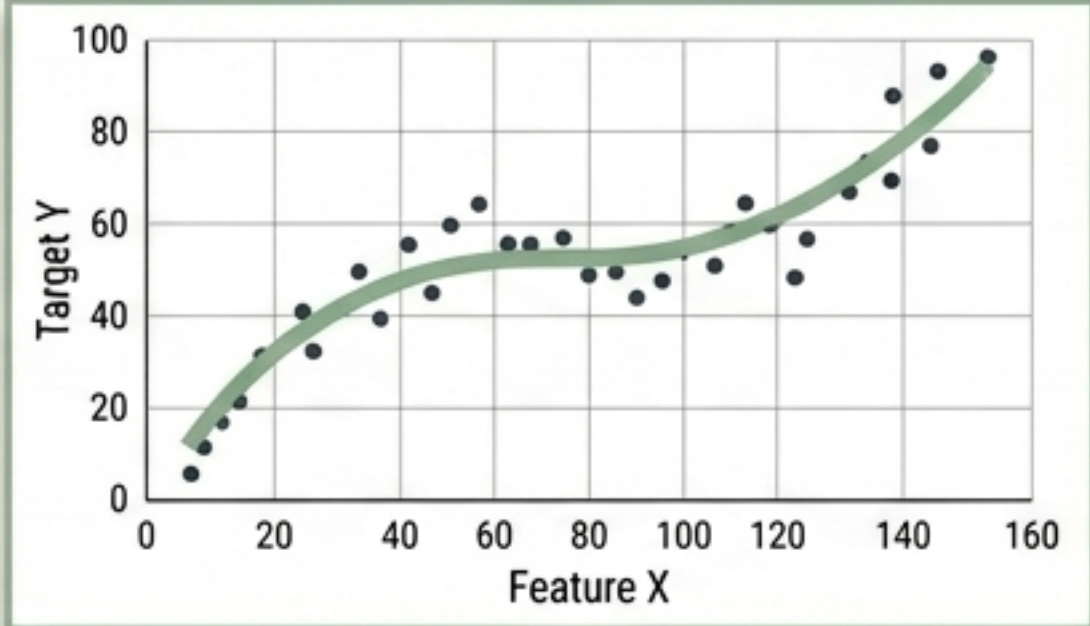
Modell-Evaluation: Die Balance zwischen Bias und Varianz

Fit Matrix

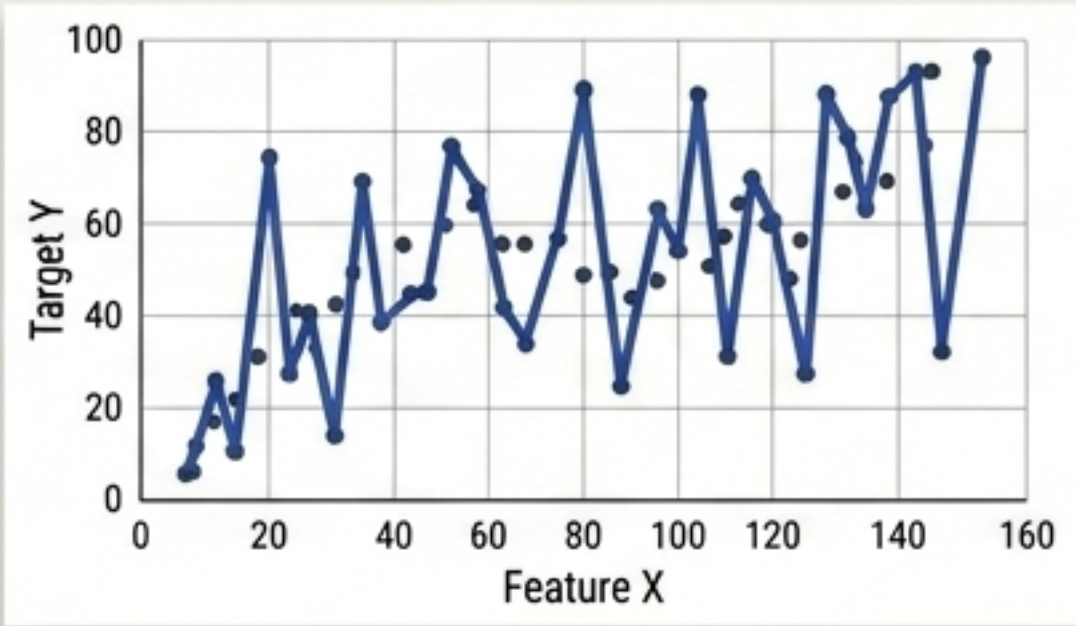
Underfitting (High Bias)



Optimal Fit

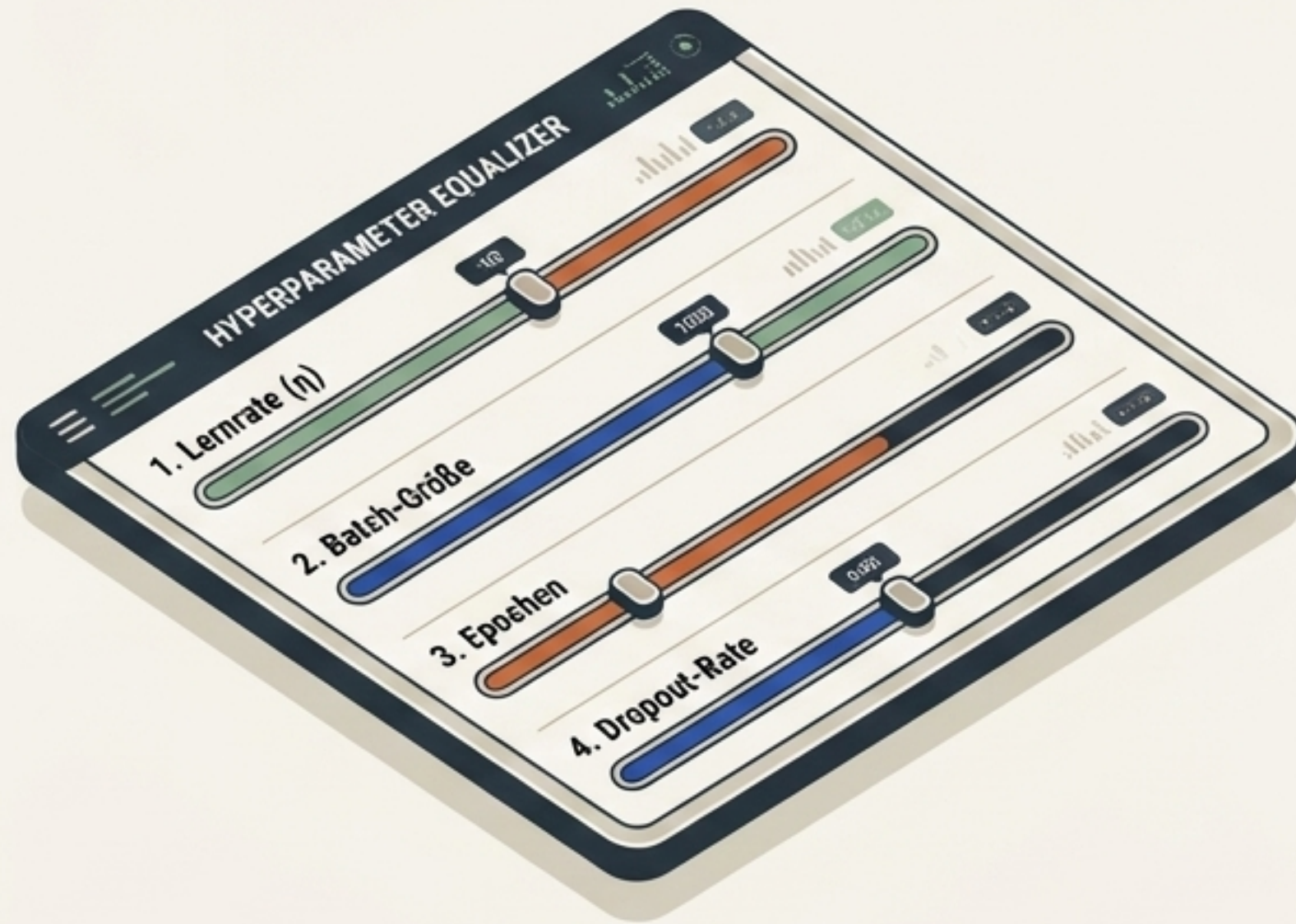


Overfitting (High Variance)



Underfitting (High Bias)		Overfitting (High Variance)	
Problem	Das Modell ist zu simpel und erfasst die zugrundeliegende Struktur nicht.	Problem	Das Modell lernt die Trainingsdaten 'auswendig' inklusive Rauschen, verliert aber die Generalisierungsfähigkeit für neue Daten.
Symptome	Schlechte Metriken auf Trainings- und Testdaten.	Symptome	Hohe Trainings-Genauigkeit, extrem abfallende Test-Genauigkeit.
Lösung	Komplexere Algorithmen, Feature Engineering.	Lösungen (Regularisierung)	Dropout (deaktiviert zufällige Neuronen), L1/L2-Regularisierung (bestraft große Gewichte), Pruning (Stutzen von Entscheidungsbäumen), mehr Validierungsdaten.

Parametertuning: Die Kalibrierung des Modells



1 Was sind Hyperparameter?

Konfigurationsvariablen, die das Verhalten des Algorithmus vor dem Training bestimmen (im Gegensatz zu Modellgewichten, die währenddessen gelernt werden).

2 Zentrale Tuning-Parameter

- **Lernrate:** Bestimmt die Schrittgröße der Gewichts Anpassung (Gradientenabstieg).
- **Baumtiefe (Max Depth):** Verhindert Overfitting bei Random Forests.
- **K-Wert:** Bestimmt die Anzahl der Nachbarn bei KNN.

3 Optimierungs-Methoden

- **Grid Search:** Systematisches Durchrechnen aller Kombinationen eines vordefinierten Rasters (sehr rechenintensiv).
- **Random Search:** Zufälliges Auswählen von Kombinationen (oft effizienter bei großen Suchräumen).

Demonstration: Typische Business-Szenarien

Business Scenarios Dashboard



1. Schätzung von Kennzahlen (Regression)

Vorhersage des Customer Lifetime Values (CLV) oder von Absatzzahlen basierend auf historischen Verkaufsdaten.



2. Entscheidungsprozesse (Klassifikation)

Churn-Prediction (Kündigungswahrscheinlichkeit) oder Betrugserkennung (Kreditkarten) via Random Forest / SVM.



3. Kundensegmentierung (Clustering)

Identifikation von Zielgruppen auf Basis von Kaufverhalten und Demografie mittels K-Means (ohne vordefinierte Labels).



4. Cross-Selling (Empfehlungsmaschinen)

Generierung von "Next Best Offer"-Empfehlungen im E-Commerce durch kollaborative Filterung oder Apriori-Algorithmen.



5. Dynamic Pricing (Reinforcement Learning)

Algorithmische Preisanpassungen in Echtzeit durch DQN, die Angebot, Nachfrage und Konkurrenzpreise als "Umgebung" betrachten.

Business to Code Translation Pipeline

[Business Goal]



[Mathematical Paradigm]



[Algorithm Choice]



[Python Library Selection]

Der Übersetzungsprozess

1. Business Frage

Welche Kunden haben das höchste Abwanderungsrisiko?



2. ML-Paradigma

Überwachtes Lernen (Historische Daten von gekündigten Kunden liegen vor).



3. Algorithmus

Random Forest Klassifikator (Robust, interpretiert Wichtigkeit einzelner Merkmale).



4. Tech-Stack

Python & Scikit-Learn.

Umkehrbeispiel

1. Business Frage

Wie fassen wir 100.000 Kunden in sinnvolle Marketing-Gruppen zusammen?



2. ML-Paradigma

Unüberwachtes Lernen.



3. Algorithmus

K-Means Clustering.



4. Tech-Stack

Python & Scikit-Learn.

Praxisübungen: Eigene Modellentwicklung

Architektur eines ML-Skripts entlang des Vorgehensmodells:

Scikit-Learn (Für Regression, Random Forest, K-Means)

1. **Init:**
`model = RandomForestClassifier()`
2. **Train:**
`model.fit(X_train, y_train)`
3. **Predict:** `y_pred = model.predict(X_test)`
4. **Eval:** `accuracy_score(y_test, y_pred)`

TensorFlow / Keras (Für Neuronale Netze / DQN)

1. **Build:**
`model = Sequential([Dense(64, activation='relu'), ...])`
2. **Compile:** Definieren von Optimierer (z.B. Adam) und Verlustfunktion (Loss).
3. **Train:** `model.fit(X, y, epochs=50)`

Ergebnisdiskussion

Bewertung anhand von RMSE (Fehler) oder R^2 (Erklärte Varianz) für Regression, bzw. F1-Score für unbalancierte Klassifikation.